Chapter 8 worked solutions

DS08-01 • Why tombstones are needed

Slots 3,4,5 initially contain 10,17,24. Deletion changes slot 4 to Deleted, not Never-used. Searching for 24 probes 3,4,5. Inserting 31 remembers reusable slot 4 but scans through to Never-used slot 6 before confirming absence, then writes at 4. If an equal key were already farther along the collision sequence, writing immediately at Deleted would create a duplicate. Although 31 is absent in this instance, insertion must satisfy the general dictionary contract.

DS08-02 • Probe-sequence coverage

Step 2 visits `1,3,5,7` and returns to 1, period 4. Step 3 visits `1,4,7,2,5,0,3,6`, period 8. Returning requires m to divide k · s. The least positive k is m/gcd(m, s), so full coverage holds exactly when gcd(m, s)=1. A noncoprime step can cycle through only part of a nonfull table. A genuinely full table is another failure case. Stop after at most m probes instead of looping indefinitely.

DS08-03 • Conditions of expectation formulas

The unsuccessful bound 1/(1−α) is 2 and 10. The successful bound ln(1/(1−α))/α is approximately 1.3863 and 2.5584. They require ideal random probe permutations, no tombstones, and the stated successful-query distribution. Linear probing follows consecutive slots and has correlated sequences and primary clustering. Tombstones add slots that do not terminate search. Current n/m alone therefore does not describe its effective occupancy or yield these exact averages.

DS08-04 • Resize oscillation

Start with capacity m and n=m/2. Insert one element to cross 1/2 and grow to 2m. Load is now near 1/4. Delete one and load is below 1/2, immediately shrinking to m. Constantly many updates can thus cause rebuilding linear in the population even under a model with linear rehashing, defeating constant amortized cost. Instead grow above 3/4 and shrink below 1/4. Growth leaves load near 3/8, and shrinkage leaves it near 1/2. A constant fraction of capacity separates these states from the next threshold, requiring Ω(n) intervening updates to pay for a Θ(n) rebuild. Also enforce a minimum small-table capacity. For randomized collision resolution, the corresponding full time guarantee is expected amortized.

Chapter 8 worked solutions (continued)

DS08-05 • Hash families, collision pairs, and quantifiers

At least one coordinate of x−y is nonzero. Fix the other coefficient; the coefficient on that nonzero coordinate has exactly one field value making the inner product zero. Pair collision probability is 1/p. Let Ixy indicate each unordered collision pair. Then τ=ΣIxy and Eτ=binom(n,2)/p. An external fixed query instead has n potential collision partners and expected colliding-key count n/p, plus bucket-access work. These are different counts.

Each of six unordered pairs collides under 11 of 121 coefficient choices, giving 66 pair-collision events divided by 121, or 6/11. Function a=b=0 contributes six collisions but is only one selected function. Universality bounds collision probability over the family for each fixed pair; it does not require every individual function to be collision-free.

Translation g_c(x)=x1+c mod p makes two keys with equal x1 and different x2 collide with probability 1. It changes bucket names without changing collision classes. The guarantee fixes S before sampling h; choosing S after seeing h changes quantifiers. The sum uses linearity of expectation without independence among all Ixy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 8 worked solutions (continued)

DS08-06

> **After deletion.** Slots 0..7 are [22:C,N,N,N,N,N,6:A,D], with n=2,d=1. Find 22 probes 6→7→0 and succeeds on probe three. Replacing slot 7 by N would falsely stop on probe two, although 22 remains at 0.
>
> **Update and insertion.** Updating 22 sees a mismatch at 6, remembers D at 7, then finds 22 at 0. Only its value changes C→C*; n=2,d=1 stay unchanged. Inserting 30 probes 6→7→0→1, proves absence at N in slot 1, then writes candidate 7; n=3,d=0. Final slots are [22:C*,N,N,N,N,N,6:A,30:Z]. Premature reuse during update would duplicate 22 at 7 and 0.
>
> **One-cycle bound.** Filling then erasing the other seven keys leaves [22,D,D,D,D,D,D,D], n=1,d=7. Find 38 probes 6→7→0→1→2→3→4→5: eight probes, no match, return absent(6). No N does not permit an infinite loop. If all eight probes see other Occupied keys and no D exists, return full.
>
> **Same-capacity rebuild.** Reinsert only live key 22 at 22 mod 8=6, yielding [N,N,N,N,N,N,22,N], n=1,d=0. Find 38 now probes 6→7. Keeping 22 at old slot 0 while clearing D to N would make find 22 fail immediately at home 6. The automatic policy tests d>m/8 before a genuinely new insertion and limits used-slot occupancy. This exercise disables that policy to retain the d=7 boundary; the assumptions must not be mixed.
>
> **Invariant.** An empty table has no live keys. A new insertion skips only O or D before its chosen slot, never N; an update leaves the location unchanged. Erasing O→D and reusing D→O introduce no N, so other live keys keep reachable prefixes. Rebuilding restores the invariant by reinserting into a new table. Writing at the first D is valid for insert_new only when the caller guarantees absence; otherwise a later duplicate must still be excluded.

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study